

A filter-based feature construction and feature selection approach for classification using Genetic Programming

Jianbin Ma^{a,*}, Xiaoying Gao^b

^a College of Information Science and Technology, Hebei Agricultural University, Baoding, 071001, China

^b School of Engineering and Computer Science, Victoria University of Wellington, Wellington, 6140, New Zealand

ARTICLE INFO

Article history:

Received 12 January 2019

Received in revised form 6 January 2020

Accepted 21 March 2020

Available online 26 March 2020

Keywords:

Genetic Programming

Feature construction

Feature selection

Classification

ABSTRACT

Feature construction and feature selection are two common pre-processing methods for classification. Genetic Programming (GP) can be used to solve feature construction and feature selection tasks due to its flexible representation. In this paper, a filter-based multiple feature construction approach using GP named FCM that stores top individuals is proposed, and a filter-based feature selection approach using GP named FS that uses correlation-based evaluation method is employed. A hybrid feature construction and feature selection approach named FCMFS that first constructs multiple features using FCM then selects effective features using FS is proposed. Experiments on nine datasets show that features selected by FS or constructed by FCM are all effective to improve the classification performance comparing with original features, and our proposed FCMFS can maintain the classification performance with smaller number of features comparing with FCM, and can obtain better classification performance with smaller number of features than FS on the majority of the nine datasets. Compared with another feature construction and feature selection approach named FSFCM that first selects features using FS then constructs features using FCM, FCMFS achieves better performance in terms of classification and the smaller number of features. The comparisons with three state-of-art techniques show that our proposed FCMFS approach can achieve better experimental results in most cases.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Classification is an important supervised machine learning method that aims to classify unknown instances into corresponding categories based on the information contained in predefined features [1,2]. So, the quality of features is a main factor that affects the classification performance [3]. Without prior knowledge, it is difficult to know which features are effective. Therefore the sufficient number of features is usually predefined, which results in many irrelevant and redundant features. Irrelevant and redundant features are not useful for classification and even reduce the classification performance [4,5]. In some real-world classification applications, available features sometimes do not have adequate discrimination ability [3], so the trained classification model cannot achieve adequate classification performance.

Feature selection method is used to select effective features and remove irrelevant or redundant features [1], and feature construction method is employed to create new higher-level features from original ones to reduce the dimensionality of features and increase the classification performance [6]. Wrapper and filter

based on different evaluation criteria. Wrapper-based approaches use the classification performance of a learning algorithm as the evaluation criterion, while information measures such as Information Gain (IG) [7], Information Gain Ratio (IGR) [3] and Correlation [8,9] are used as evaluation criteria for filter-based approaches. Wrapper-based approaches are learning algorithms dependent, and in general these methods can achieve better classification performance than filter-based approaches. Since no learning algorithm is involved in the evaluation measures, filter-based approaches are faster and the classification models are more general than wrapper-based approaches [10,11]. Moreover, experiments show that our proposed filter-based feature construction and feature selection method (FCWFS) can also achieve better classification performance than a wrapper-based feature construction method.

To address feature construction and feature selection problems, efficient global search algorithms are needed. Genetic Programming (GP) [12,13] offers flexible pattern representations such as trees, and uses any kind of logical and mathematical expressions inside the representations [14]. These expressions can transform original features into new higher-level constructed features, and can also be used to select effective features. Therefore, GP can be used to solve feature selection [15–17] and feature construction [3,10] tasks.

* Corresponding author.

E-mail address: majianbin@hebau.edu.cn (J. Ma).

Feature construction approaches are used to transform the original feature space to another higher-level feature space. In general, GP can be used to construct single features. Otero et al. (2002) [3] used IGR as the fitness function and constructed single feature. Muharram et al. (2005) [18] proposed a single feature construction method that employed information gain, gini index, chisquare, and a combination of information gain and gini index as fitness functions. Guo et al. [19,20] proposed similar single feature construction methods, and the difference is that these methods use Fisher criterion as fitness functions. Since single constructed features do not have adequate discrimination ability for classification, multiple feature construction approaches are investigated to improve the classification performance. Neshatian et al. (2012) [10] used a fitness function that maximized the purity of class intervals and constructed the same number of features as the number of classes. Krawiec (2002) [21] proposed an archive-based multiple feature construction method that stored useful individuals during evolutionary run. Ahmed et al. (2014) [22] divided the best individual to all possible sub-trees that were transformed into multiple features. Moreover, cooperative coevolution strategy [23–26] that created multiple cocurrent populations was used to construct multiple features.

With the evolution of GP, many excellent individuals are often lost. In order to preserve effective constructed features during a GP run, a multiple feature construction approach (FCM) that stores top β individuals is proposed in our previous work [27]. However, how to set the parameter β is a problem. In this work, we investigate the impact of parameter β on the experimental results and set a β value that is as large as possible to maintain the classification performance. Thus, redundant features may be produced in the constructed features. Therefore, we employ a GP-based feature selection approach (FS) that uses correlation-based method to reduce feature redundancy and increase feature relevancy. This approach is named as FCMFS that first uses FCM to perform feature construction and then uses FS to perform feature selection.

1.1. Goals

To facilitate comparison, both feature selection and feature construction approaches use standard GP. The overall goal of this paper is to propose a feature construction and feature selection approach (FCMFS), which first constructs multiple features using FCM then selects effective constructed features using FS, and to investigate the effectiveness of our proposed FCMFS by comparing it with other feature processing methods. In order to achieve the overall goal, the following four objectives will be investigated.

Objective 1. Propose a filter-based multiple feature construction approach using GP (FCM) and a filter-based feature selection approach using GP (FS), and investigate whether features selected by FS or constructed by FCM can obtain better classification performance than original features.

Objective 2. Develop a feature construction and feature selection approach named FCMFS, and investigate whether FCMFS can obtain equivalent classification performance with a smaller number of features comparing with FCM, and whether FCMFS can achieve better classification performance and fewer features than FS on the nine datasets.

Objective 3. Investigate another feature construction and feature selection method that first selects features using FS then constructs features using FCM named FSFCM, and verify whether FCMFS can achieve better classification performance than FSFCM.

Objective 4. Investigate whether our proposed FCMFS can achieve better performance than three state-of-art methods including one wrapper-based feature construction method [28], one filter-based multiple feature construction method [10] and one single-stage feature construction and feature selection method [29].

1.2. Organization

The rest of the paper is arranged as follows. The next section describes background information involved in this paper. Section 3 presents the GP based feature construction and feature selection approaches. Section 4 describes the experimental design. Section 5 presents the experimental results with discussions. Section 6 provides conclusions and future directions.

2. Background

2.1. Genetic programming

The evolutionary computation (EC) techniques are inspired by Darwin's theory of evolution [30]. Genetic programming (GP) [12, 31], Genetic algorithm (GA), Particle swarm optimization (PSO), Ant colony optimization (ACO) are effective EC algorithms due to their global search ability. In addition, some new EC algorithms, such as extremal optimization (EO) algorithms, are used to solve optimization problems [32,33]. Addressing multi-objective optimization problems using EC algorithms are getting more attentions [34–36]. The main differences between all kinds of EC algorithms are their individual representation methods and evolution strategy.

GP is the evolution of computer programs. The commonly used representation method is based on tree-like structures, although other methods exist [13,37]. GP individuals are usually treated as trees, where leaf nodes are randomly generated from the terminal set (constants or variables), and internal nodes are randomly generated from the function set (arithmetic operators and logical operators). GP randomly generates an initial population which consists of individuals. A fitness function is used to evaluate the fitness of an individual. The individuals with higher fitness values have a bigger probability to survive. Selection method is used to select individuals from the population to the next population based on the fitness of individuals. Genetic operators including reproduction, crossover and mutation are applied to produce new individuals with specific probabilities. GP loop executes the above steps until the termination criterion is reached and presents the best individual in the population as the output.

2.2. Information gain ratio (IGR)

IGR is used to measure the quality of a feature [38], and is the splitting criterion of the C4.5 decision tree algorithm [39]. IGR takes the number and the size of branches into account when choosing a feature instead of only taking the intrinsic information into account, and overcomes the overfitting problem of Information Gain (IG) method to some extent. This measure criterion is fast and the value of IGR can be computed via a single scan of the training set [3]. IGR is commonly used as a fitness function to evaluate the effectiveness of a single feature [3,40].

2.3. Correlation-based feature evaluation method

IGR is only used to evaluate a single feature. It cannot be used to evaluate a feature subset. The correlation-based feature evaluation method was proposed by Hall [41–43] to evaluate and rank feature subsets rather than individual features on the basis of the following hypothesis: "Good feature subsets contain features highly correlated with classes, yet uncorrelated to each other". The correlation-based feature evaluation method is a heuristic for evaluation the merit of feature subsets. The following formula gives the heuristic merit of a feature subset S consisting of k features.

$$\text{Merit}_S = \frac{k\overline{r_{cf}}}{\sqrt{k + k(k-1)\overline{r_{ff}}}} \quad (1)$$

where $Merit_S$ is the heuristic “merit” of a feature subset S containing k features. $\bar{r}_{cf}$ is the average feature-class correlation. $\bar{r}_{ff}$ is the average feature-feature inter-correlation.

Based on Formula (1), irrelevant features are removed from feature sets due to low correlation with classes. Redundant features are screened out as they will be highly correlated with one or more of the remaining features [41].

2.4. Research work on feature selection using GP

GA [44,45], ACO [46] and PSO [1,47] are widely used in feature selection. Compared with GA, ACO and PSO, there are fewer works on GP for feature selection. In feature selection using GP, GP is usually represented as tree-like structures. The terminal (leaf) nodes of a tree are the selected features. GP is used to search for the best feature subsets.

In filter-based approaches, GP is mainly used for searching for the best feature subsets. Neshatian (2009) [48] proposed a GP system for measuring the relevance of feature subsets in binary classification tasks. The system could rank the feature subsets for feature selection tasks. A multiple-objective feature selection method using GP was proposed in Neshatian (2009) [49], which improved standard GP to explore larger feature subsets and provided solutions from Pareto-front points. Neshatian (2012) [50] improved relevance measures to evaluate the selected features and the class label, and used GP for feature selection. In filter-based feature selection approach using GP, works have been done on either improving standard GP algorithm or improving evaluation criteria of feature subsets. In our work, we use standard GP algorithm and employ correlation-based feature evaluation method to evaluate the effectiveness of the feature subsets.

In most wrapper-based approaches, GP is used for both searching for the best feature subsets and learning a classifier [51]. Muni (2006) [52] proposed a method that simultaneously selected a good feature subset and constructed a classifier using the selected features, and presented two new crossover operators to improve the performance of feature selection. Purohit (2010) proposed a similar method as Muni [52], where a modified crossover operator was proposed to find the best possible subtree crossover point and obtain higher fitness. To overcome the effect of weakly relevant or irrelevant features on the feature selection, two-stage wrapper-based feature selection approaches using GP have been investigated. In the first stage, the features are ranked by information measures. Then, in the second stage, GP is used to search for effective feature subsets from the top features. Typical researches are shown in [15,53–55]

2.5. Research work on feature construction using GP

GP is suitable for handling feature construction tasks due to its flexible representation. According to different evaluation functions, feature construction are divided into filter-based and wrapper-based approaches.

In the filter-based approaches, GP optimizes a population of computer programs according to information measures such as information gain (IG) [7], information gain ratio (IGR) [3] and Fisher criterion [19,22,56]. Otero et al. (2002) [3] proposed a single feature construction method which used IGR as a fitness function, and experimented on the augmented feature set including original features and a constructed feature. Experimental results show that the original features combined with the constructed feature can increase the classification performance of C4.5. Muharram et al. (2005) [18] proposed a single feature construction method. Four different fitness functions including information gain, gini index, chisquare, and a combination of information gain and gini index were adopted. Experiments on

the augmented features were done on decision tree algorithms and multilayer perceptron. Experimental results show that the constructed feature can improve the classification performance without biasing the fitness functions. Similar single feature construction method was proposed by Guo [19,20]. The difference between these methods depends on which fitness function is used. Neshatian et al. (2012) [10] proposed a multiple feature construction approach for symbolic learning classifiers. A fitness function that maximized the purity of class intervals was proposed. In this method, GP needs to be run multiple times. The number of GP runs equals to the number of classes in the dataset.

In the wrapper-based feature construction approaches, the constructed features are evaluated by the classification performance, and the approaches are classifier dependent. In general, each individual constructs only one single feature [28]. However, researchers try to find ways to construct multiple features to improve the classification ability of the learning algorithm. One multiple feature construction method is to use cooperative co-evolution strategy [23–26]. The basic ideas of this method is to create multiple cocurrent populations, and then find optimal individuals. Another multiple feature construction method is to store the best individuals. Krawiec (2002) [21] proposed a multiple feature construction method that stored useful individuals during evolutionary run. The method provided predefined number of evolution features and hidden features. Hidden features stored most effective features based on the feature utility function which computed the relative usage of original features in the process of decision tree induction. The main problem of the above two multiple feature construction methods is predefining specific number of constructed features. However, the number of constructed features depends on specific datasets and the parameter value is hard to set. The third multiple feature construction method is to separate GP trees into multiple sub-trees. Each sub-tree corresponds to a constructed feature. Ahmed et al. (2014) [22] developed a multiple feature construction method for classification based on the ideas of dividing the best individual to all possible sub-trees. This method does not require pre-fixing the number of features, however, the number of features constructed by this method is related to the size of the program. In general, the feature dimension of this method is still high, and there will be a lot of redundant features.

2.6. Research work on feature construction and feature selection using GP

To avoid generating too many redundant features, two-stage feature construction and feature selection method which executes feature construction and feature selection sequentially is proposed.

Kamath (2014) [57] proposed a framework that first constructed multiple features using GP and then selected effective feature subsets for classification using GA. In the feature construction stage, the l best individuals are stored into the *hall of fame* in every generations, and randomly select m individuals to seed the population in the next generation. This method did not involve how to set the value of l and m , and the setting of parameters often depends on specific problems. This method just involved the classification problem of biological sequences. The feature construction stage is wrapper-based and the feature selection stage is filter-based.

Smith (2005) [40] first used GP to construct multiple features, and the number of constructed features was the same as that of original features, then employed GA to select effective features. Feature construction method presents a novel population initiation method. Each individual in the population consists of terminal nodes and sub-trees of a GP tree. Terminal nodes is

the corresponding original features, and the sub-trees are transformed to sub-tree features. The problem of this method is how to determine the proportion of terminal features to sub-tree features. Both feature construction and feature selection methods are wrapper-based.

Hart (2017) [8] proposed a feature construction and feature selection method that first constructed predefined number of features by running GP algorithm multiple times, and then used GA to remove redundant features. The number of constructed features is the same as that of the GP runs. The feature construction is somewhat like the method proposed by Guo (2005) [56]. However, multiple GP runs trend to reach the same solution with GP converging [10], and the multiple constructed features are highly correlated.

A single-stage feature construction and feature selection approach was proposed by Tran (2016) [2] which used leaf nodes of a GP tree to select effective original features and used the GP tree to construct a new higher-level feature, i.e., performing feature construction and feature selection using GP at the same time. Tran (2016) [29] proposed another single-stage feature construction and feature selection method that constructed multiple features based on multiple trees representation and also chose terminal nodes as selected features.

To the best of our knowledge, there is no research involving filter-based feature construction and feature selection approaches using GP. The previous studies have not verified whether two-stage feature construction and feature selection approaches can really achieve better performance than feature construction and feature selection only, and whether the order of feature selection and feature construction affect the classification performance of two-stage feature construction and feature selection approaches. In this paper, a lot of experiments have been done on (1) a filter-based multiple feature construction approach using GP (FCM) and a filter-based feature selection approach using GP (FS), (2) a two-stage feature construction and feature selection approach using GP, i.e., FCMFS and an investigation on swapping the order of the two stages, (3) comparisons of different feature processing methods on a large number of datasets and (4) comparisons of our proposed FCMFS with benchmark techniques.

3. Methodology

We use standard GP representation methods to solve feature construction and feature selection problems. The individuals are represented as a tree-like structure. The leaf nodes of an individual are derived from original features, constructed features or selected features randomly according to different feature processing methods which are described as follows. The internal nodes are functions that come from a function set. Genetic operators, including reproduction, crossover and mutation, are the same as traditional methods.

In this paper, a multiple feature construction method using GP (FCM) that stores top individuals during a GP run is proposed. A GP-based feature selection approach (FS) that uses correlation-based evaluation method to reduce feature redundancy and increase feature relevancy is employed. The basic idea of two-stage feature construction and feature selection approaches is that feature construction and feature selection are carried out sequentially. According to the order of feature construction and feature selection, there are two two-stage feature construction and feature selection approaches, that is, first feature construction then feature selection algorithm (FCMFS) and first feature selection then feature construction algorithm (FSFCM).

The following describes five algorithms, i.e., single feature construction method (FCS), multiple feature construction method (FCM), feature selection method (FS), first feature selection then

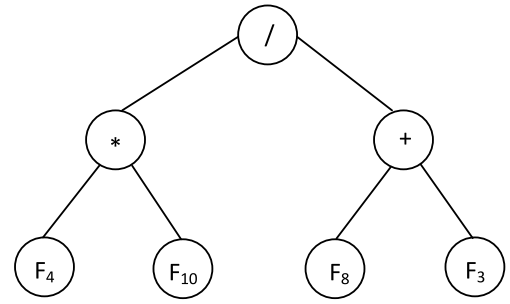


Fig. 1. An example of GP tree.

feature construction approach (FSFCM) and first feature construction then feature selection approach (FCMFS). The conventional FCS method plus the method using original features named ALL are chosen as baselines to compare with proposed FCMFS. FCM, FS and FSFCM are also compared with our proposed FCMFS to verify the performance of proposed FCMFS.

3.1. Single feature construction method (FCS)

FCS aims to construct single feature. GP presents the best individual (GP tree) in the population as the final output. A GP tree in the population can be regarded as an arithmetic expression of original features, and is treated as single constructed feature. An example of a GP tree is shown in Fig. 1. The function of the constructed feature F_c using original features $\{F_3, F_4, F_8, F_{10}\}$ is: $F_c = (F_4 * F_{10}) / (F_8 + F_3)$.

An individual is better when its fitness value is lower. So, we use Formula (2) to evaluate constructed features.

$$fitness_construction = (1 - IGR) \quad (2)$$

where IGR is the information gain ratio of a constructed feature.

3.2. Multiple feature construction method (FCM)

With the evolution of GP, some excellent individuals may occur in certain generations, however, these individuals may be lost with the subsequent evolution [57,58], and the best individual is not necessarily the best. So, we proposed a method to store top β individuals $I_s = \{i_1, i_2, \dots, i_\beta\}$ in a storage pool. For each generation, the individuals in the GP population are compared with the β individuals in the storage pool. If there are better individuals in the GP population than the individuals in the storage pool, the individuals in the storage pool are replaced by the individuals in the population. In this way, the top β individuals in all the individuals generated by the algorithm are stored in the storage pool.

How to set the value of β is discussed in Section 5.3. To maintain the classification performance and accomplish feature selection tasks, the parameter value of β is set to 20.

3.3. Feature selection method (FS)

The features appearing at terminal nodes in a GP tree is the selected features. As shown in Fig. 1, $\{F_3, F_4, F_8, F_{10}\}$ are the selected features. We use correlation-based evaluation method to reduce feature redundancy and increase feature relevancy. The fitness function for selected features is shown in Formula (3).

$$fitness_selection = (1 - CORR) \quad (3)$$

where CORR is the correlation of selected features and classes. The calculation method of correlation-based evaluation method is shown in Section 2.3.

3.4. First feature selection then feature construction approach (FS-FCM)

The idea of FSFCM is to first select effective features and remove redundant features using FS, and then construct multiple features using FCM. The purpose of the algorithm is to investigate whether the redundant and irrelevant features in original features affect feature construction performance.

First, the original features $F_o = \{f_1, f_2, \dots, f_n\}$ are reduced to a smaller feature subset $(F_o)_s = \{f_{s1}, f_{s2}, \dots, f_{st}\}$ by FS, where n is the number of original features, t is the number of selected features, f_j denotes the j th original feature, and f_{si} denotes the i th selected feature. Then, in the second stage, multiple features $(F_o)_{scm} = \{f_{sc1}, f_{sc2}, \dots, f_{scm}\}$ are constructed by FCM and the terminal nodes of GP are generated from $(F_o)_s$, where m is the number of constructed features and f_{scj} denotes the j th constructed feature. This algorithm needs two GP runs, the first GP run select effective features, then the second GP run constructs multiple features.

3.5. First feature construction then feature selection approach (FCMFS)

The idea of the FCMFS algorithm is to first construct multiple features using FCM, and then select effective features and remove redundant features using FS.

In feature construction stage, suppose constructed features are represented as $F_{cm} = \{f_{c1}, f_{c2}, \dots, f_{cm}\}$ which is generated from original features $F_o = \{f_1, f_2, \dots, f_n\}$, where f_{ci} denotes the i th constructed feature. Then, in feature selection stage, the terminal nodes of an individual in the GP population are chosen from F_{cm} , and F_{cm} will be reduced to $(F_{cm})_s$ after feature selection is performed.

Proposed FCMFS needs two separate GP runs rather than one. The first GP run constructs multiple features, in this paper, the number of constructed features is set to 20. The second GP run only selects effective features from the constructed features, that is, the constructed features become the new terminals. The Pseudo-Code of FCMFS algorithm is shown in Algorithm 1. In the algorithm, p denotes the polulation size of GP. fit_i denotes the fitness of an individual i . $Fit_s = \{fit_1, fit_2, \dots, fit_\beta\}$ denotes the fitness of individuals $I_s = \{i_1, i_2, \dots, i_\beta\}$ in the storage pool and the set Fit_s is sorted according to ascending order. In the algorithm, the first 12 steps is the feature construction stage, and steps 14 to 22 is the feature selection stage.

After the above feature processing methods, the testing set is transformed into a new testing set, and then learning algorithms are used to evaluate the classification performance of the new testing set.

Our work that uses GP to solve feature selection and feature construction tasks does not involve multi-objective optimization problems. There are no multiple conflicting objectives for feature construction. Some researchers focused on multi-objective feature selection problems [1], however, correlation performance is our more focused optimization objective.

4. Experimental design

4.1. Benchmark techniques

To verify the effectiveness of our proposed FCMFS, three state-of-art techniques including one wrapper-based feature construction method [28], one filter-based multiple feature construction method [10] and one single-stage feature construction and feature selection method [29] are chosen for comparison.

Algorithm 1: Pseudo-Code of FCMFS algorithm

Input: $\mathcal{D}$: a training set
Output: $(F_{cm})_s$: the set of constructed and selected features

```

1 Randomly initialize the individuals in the population of GP
2 while Maximum Generations is not reached do
3   for  $j = 1$  to  $p$  do
4     Calculate  $fit_j$  of the individual  $j$  on training set  $\mathcal{D}$ 
       according to Formula 2
5     if  $fit_j < fit_\beta$  then
6       Insert  $i_j$  at an appropriate position in  $I_s$ 
       //store  $i_j$  in the storage pool
7     end
8   end
9   Select individuals using tournament method
10  Perform Reproduction, Crossover and Mutation
    operations
11 end
12 Return constructed features  $F_{cm} = \{f_{c1}, f_{c2}, \dots, f_{c\beta}\}$ 
    //feature construction stage
13 Transform the training set  $\mathcal{D}$  into  $\mathcal{D}_c$  based on the
    constructed features
14 Randomly initialize the individuals in the population of GP
15 while Maximum Generations is not reached do
16   for  $j = 1$  to  $p$  do
17     Calculate  $fit_j$  of the individual  $j$  on training set  $\mathcal{D}_c$ 
       according to Formula 3
18   end
19   Select individuals using tournament method
20   Perform Reproduction, Crossover and Mutation
    operations
21 end
22 Return selected features  $(F_{cm})_s$  //feature selection stage

```

The first is a conventional wrapper-based feature construction method using GP which constructs single feature, i.e., one GP run only outputs the best individual and the fitness function uses classification performance of a learning algorithm. The parameter setting of GP and the experimental design is the same as FCMFS. This method is named as FCW.

The second is one filter-based multiple feature construction method which is proposed by Nethatian (2012) [10]. This method uses a fitness function that maximizes the purity of class intervals and needs multiple GP runs. Each GP run constructs one feature. The number of constructed features is the same as the classes of a dataset. This method is named as FCMMR. More details can be seen in the literature [10].

The third is one single-stage feature construction and feature selection approach which is proposed by Tran (2016) [29]. The basic idea of this approach is that feature construction and feature selection are carried out at the same time. This method only needs one GP run. The leaf nodes of a GP tree are selected as features. Each GP tree can be constructed as a feature. The method in literature [29] is wrapper-based. To compare fairly, we modified this method and developed a filter-based approach. This method is named SFCFS. Take the GP tree in Fig. 1 as an example, SFCFS uses the *fitness_selection* in Formula (3) to evaluate the effectiveness of the individuals, that is, to compute the correlation of the feature set $F_{ct} = \{F_c, F_4, F_{10}, F_8, F_3\}$ and the classes, where F_{ct} is the constructed feature F_c augmented by the terminal nodes $F_t = \{F_4, F_{10}, F_8, F_3\}$.

4.2. Datasets and parameter settings

To evaluate the effectiveness of our proposed feature construction and feature selection approaches, nine databases involving

Table 1
Description of the nine datasets.

Dataset	#features	#instances	#classes
Wine	13	178	3
Sonar	60	208	2
wdbc	30	569	2
HillValley	100	606	2
ionosphere	34	351	2
Balance-scale	4	625	3
iris	4	150	3
Thyroid Disease	5	215	3
WBC-Original	9	683	2

classification problems shown in Table 1 have been collected from UCI machine learning repository [59]. We chose these nine datasets for two reasons: (1) All these datasets have been used in other literatures [3,10,11] on feature construction, and first five datasets [1,60] have also been used in feature selection. These datasets allow us to compare our approach with other work. (2) They have different number of features and different number of instances, and are used as the representative samples of the problems.

In order to get more convincing experimental results, each dataset in Table 1 is randomly divided into 30 different training sets and test sets using 30 different random seeds [2], 70% as the training set, and 30% as the test set [47]. So, all the GP based feature construction and feature selection algorithms described above have been run 30 times on 30 different training sets and test sets independently to avoid the effect of stochastic characteristics of GP and dataset partition. The classification accuracy is used to evaluate the experimental results. The best, average, and standard deviation of the 30 different experimental results are obtained. In the following experiments, five algorithms in Section 3 will be tested on nine datasets separately. FCMFS and FSFCM need two GP runs, and other three algorithms need one GP run. So a total of (3algorithms*9datasets+2algorithms*2*9datasets)*30GP=1890GP is run. Comparisons with three benchmarks need more GP runs.

The ECJ library [61] is used to run GP. The parameter settings of GP are shown in Table 2. The function set of GP is arithmetic operators (+, −, ×, %). The % operator is protected, that is, the % operator returns zero when it is divided by zero. The terminal set of GP is the produced features corresponding to different methods. The values of features are inputted into GP trees. For feature construction method, the output of a GP tree is the value of the new constructed feature. For feature selection method, we do not care about the output of a GP tree. The original datasets are transformed to new datasets according to the features constructed and selected by GP.

K-nearest Neighbors(KNN, K=5) and Decision Tree(DT) learning algorithms are used to evaluate the performance of the methods in Section 3. The C4.5[38] version is used as decision tree algorithm. Weka package [9] is employed to run the two learning algorithms. The value of parameter β is set to 20, and the impacts of parameter β on the algorithms are discussed in Section 5.3.

To test whether different feature processing methods are significantly different, the Friedman test with significance level 0.05 followed by the Wilcoxon–Nemenyi–McDonald–Thompson Post-hoc test [62] was used to evaluate the experimental results. The R code of Friedman test in [63] was used.

The performance differences between paired methods were calculated. Suppose the average accuracies of method M_x on nine datasets are denoted by $M_x = (a_{x1}, a_{x2}, \dots, a_{x9})$, and a_{xi} denotes the average accuracy of method M_x on the i th dataset. Then, the differences between M_x and M_y is $M_x - M_y = (a_{x1} - a_{y1}, a_{x2} - a_{y2}, \dots, a_{x9} - a_{y9})$. In the following experiments, the boxplots of

Table 2
Parameter setting of GP.

Parameters	Parameter value
Function set	+, −, ×, %(protected division)
Terminal set	Original features F_o , constructed features F_{cm} or selected features $(F_o)_s$
Population size	500
Number of generations	50
Initialization	Ramped half-and-half
Mutation method	Random subtree creation
Selection method	Tournament method
Maximum tree depth	17
Mutation probability	10%
Cross-over probability	90%
Reproduction probability	10%

performance differences between paired methods were drawn to help us understand which methods were more effective, and the Post-hoc analysis for Friedman test were marked using different colors on the boxplots to show whether two methods are significantly different. Boxplot is a method for graphically depicting the degree of dispersion and skewness of the data through quartiles [64]. If the Post-hoc for Friedman test with $p - value \leq 0.05$, the boxplots are marked green, which shows that the performances of paired methods are significantly different. If the Post-hoc for Friedman test with $0.05 < p - value \leq 0.4$, the boxplots are marked yellow, which shows that the performances of paired methods on most datasets are different.

5. Experimental results and discussions

We arranged the following experiments. Firstly FCMFS is compared with three benchmark techniques (FCW, FCMMR and SFCFS) to verify the effectiveness of the proposed FCMFS. Secondly, the effectiveness of six feature processing methods in Section 3 is compared. (1) FCMFS is compared with two baselines(ALL, FCS) to further verify the effectiveness of the proposed FCMFS, (2) FCM and FS are compared with ALL to verify the effectiveness of FCM and FS methods, and (3) FCMFS is compared with FCM and FS methods respectively to verify whether the combination of FCM and FS, i.e., FCMFS can obtain better performance than individual FCM and FS, and is compared with FSFCM to verify whether swapping the order of FCM and FS can achieve different results. Thirdly, the impacts of parameter β on FCM is experimented.

5.1. Comparisons between FCMFS with three benchmark techniques (FCW, FCMMR and SFCFS)

This section compares the experimental results of proposed FCMFS and three benchmark techniques described in Section 4.1. The experimental results of FCMFS, FCW, FCMMR and SFCFS are shown in Table 3.

We define the following terms to facilitate readability. A-KNN represents the average accuracy of 30 independent runs using KNN learning algorithm. A-DT represents the average accuracy of 30 independent runs using DT learning algorithm. In Table 3, the column “#F” is the average number of features. “B” represents the best and “A±Std” represents the average and standard deviation of the accuracy of the 30 independent runs. The best average accuracy of each dataset on each learning algorithm is marked as bold.

The boxplots of performance differences between FCMFS and three benchmark techniques marked by Post-hoc analysis for Friedman test are shown in Fig. 2. The Friedman significance test shows that there are significantly differences between FCMFS and three benchmarks with $P - value = 0.02222$ in KNN learning algorithm and $P - value = 0.002922$ in DT learning algorithm. The

Table 3

The experimental results of FCMFS, FCW, FCMMR and SFCFS.

Dataset	Methods	#F	B-KNN	A $\pm$ Std-KNN	B-DT	A $\pm$ Std-DT
Wine	FCMFS	4.30	100	94.17 $\pm$ 3.62	97.92	91.94 $\pm$ 2.98
	FCW	1	97.92	92.22 $\pm$ 3.57	95.83	90.00 $\pm$ 4.11
	FCMMR	3	100	93.06 $\pm$ 4.45	97.92	91.89 $\pm$ 3.94
	SFCFS	13.73	81.67	95.76 $\pm$ 3.93	97.92	87.57 $\pm$ 6.31
Sonar	FCMFS	5.50	85	73.78 $\pm$ 6.84	86.67	72.61 $\pm$ 7.21
	FCW	1	85	68.56 $\pm$ 7.87	88.33	71.56 $\pm$ 7.58
	FCMMR	2	81.67	68.17 $\pm$ 8.03	80	68.61 $\pm$ 8.80
	SFCFS	13.73	81.67	73.28 $\pm$ 5.45	86.67	69.33 $\pm$ 8.45
Wdbc	FCMFS	5.03	98.21	96.31 $\pm$ 1.13	98.21	95.75 $\pm$ 1.50
	FCW	1	98.21	95.32 $\pm$ 2.00	97.62	95.54 $\pm$ 1.56
	FCMMR	2	98.21	92.54 $\pm$ 4.70	98.21	94.54 $\pm$ 2.52
	SFCFS	7.30	98.21	95.32 $\pm$ 1.92	98.21	94.33 $\pm$ 1.94
HillValley	FCMFS	3.28	99.72	98.04 $\pm$ 2.62	100	99.00 $\pm$ 0.80
	FCW	1	100	99.56 $\pm$ 0.58	100	99.52 $\pm$ 0.62
	FCMMR	2	99.72	96.13 $\pm$ 3.77	99.72	98.45 $\pm$ 1.47
	SFCFS	3	62.81	58.33 $\pm$ 2.76	98.35	94.88 $\pm$ 2.41
ionosphere	FCMFS	4.60	94.12	88.89 $\pm$ 2.72	97.06	89.54 $\pm$ 3.27
	FCW	1	95.10	88.43 $\pm$ 3.12	96.08	89.18 $\pm$ 3.39
	FCMMR	2	86.27	75.03 $\pm$ 7.69	88.24	75.07 $\pm$ 8.32
	SFCFS	8.33	91.18	84.51 $\pm$ 3.10	94.12	89.05 $\pm$ 3.35
Balance-scale	FCMFS	2.10	100	99.30 $\pm$ 3.21	100	99.26 $\pm$ 3.08
	FCW	1	100	98.61 $\pm$ 2.37	100	98.61 $\pm$ 2.73
	FCMMR	3	100	99.17 $\pm$ 1.45	100	99.38 $\pm$ 1.37
	SFCFS	5	91.11	87.50 $\pm$ 2.23	100	99.06 $\pm$ 2.53
iris	FCMFS	4.63	100	95.41 $\pm$ 2.92	97.78	92.30 $\pm$ 4.28
	FCW	1	100	94.81 $\pm$ 2.77	97.78	91.56 $\pm$ 4.19
	FCMMR	3	100	94.81 $\pm$ 3.41	100	92.07 $\pm$ 3.61
	SFCFS	4.87	100	95.26 $\pm$ 2.61	100	93.70 $\pm$ 3.54
Thyroid	FCMFS	4.47	98.41	94.60 $\pm$ 2.34	96.83	94.02 $\pm$ 2.95
	FCW	1	98.41	94.29 $\pm$ 2.48	96.83	93.44 $\pm$ 2.68
	FCMMR	3	96.83	93.54 $\pm$ 3.50	96.83	93.39 $\pm$ 3.01
	SFCFS	5.90	98.41	91.64 $\pm$ 3.07	96.83	92.22 $\pm$ 3.72
WBC	FCMFS	4.87	98.41	94.73 $\pm$ 1.26	98.55	96.05 $\pm$ 1.45
	FCW	1	97.10	95.33 $\pm$ 1.65	99.03	95.97 $\pm$ 1.40
	FCMMR	2	98.55	94.68 $\pm$ 1.32	98.07	95.48 $\pm$ 1.43
	SFCFS	9.53	98.55	96.30 $\pm$ 1.43	98.07	95.43 $\pm$ 1.47

Post-hoc analysis for Friedman test marked on boxplots shows that the performance difference between FCMFS and FCMMR is significant in KNN learning algorithm, and the performance differences between FCMFS and FCMMR, FCMFS and SFCFS are significant in DT learning algorithm.

Though FCMFS cannot achieve significantly better classification performance comparing with FCW, according to Table 3, FCMFS still has advantages over FCW. FCMFS achieves better classification performance comparing with FCW in both KNN and DT learning algorithms on almost all the datasets except for HillValley and Balance-scale datasets. Moreover, compared with wrapper-based FCW, filter-based FCMFS needs less training time, and the classification models are more general [10,11].

FCMMR proposed by Neshatian (2012) [10] used rule inducers and decision tree learners in the experiments. This method was also experimented on KNN learning algorithm and compared with our proposed FCMFS in this paper. The experiments verify that the classification performances of FCMFS are better than those of FCMMR on almost all the datasets in both KNN and DT learning algorithms. Moreover, FCMMR needs multiple GP runs, and the number of GP runs equals to the number of dataset's classes. Our proposed FCMFS needs two GP runs, i.e., feature construction and feature selection stage. In general, FCMFS needs less GP runs and can achieve better classification performance than FCMMR.

As shown in Table 3, FCMFS is particularly better than SFCFS on the HillValley dataset. A-KNN using SFCFS is just 58.33% accuracy, however, A-KNN using FCMFS can obtain more than 98% accuracy. Moreover, FCMFS achieves better performance than SFCFS not only on classification performance but also on the

number of features. According to Table 3, FCMFS can construct equivalent or smaller number of features comparing with SFCFS.

Compared with three state-of-art techniques, our proposed FCMFS has obvious advantages, which shows that FCMFS is effective, and can obtain a feature subset with good discrimination ability under the condition of as few features as possible.

5.2. Comparisons of six feature processing methods

Table 4 shows the experimental results of different feature processing methods described in Section 3. The column of **Subset** is the feature subset produced by different methods described in Section 3.

5.2.1. Comparisons between FCMFS and two baselines (ALL, FCS)

The boxplots of performance differences between FCMFS, FCS and ALL marked by Post-hoc analysis for Friedman test are shown in Fig. 3. As shown in Fig. 3, there are significant differences between three methods with $P\text{-value}=0.0257$ in KNN learning algorithm and $P\text{-value}=0.01287$ in DT learning algorithm. The Post-hoc analysis shows us that the performance differences between FCMFS and two baselines (FCS and ALL) are significant in KNN learning algorithm. For DT learning algorithm, the performance difference between FCMFS and ALL is significant, and the performance difference between FCMFS and FCS is borderline significant.

From the experimental results in Table 4, we can see that FCMFS is better than ALL on almost all the datasets in both KNN and DT learning algorithms. ALL using KNN and DT learning

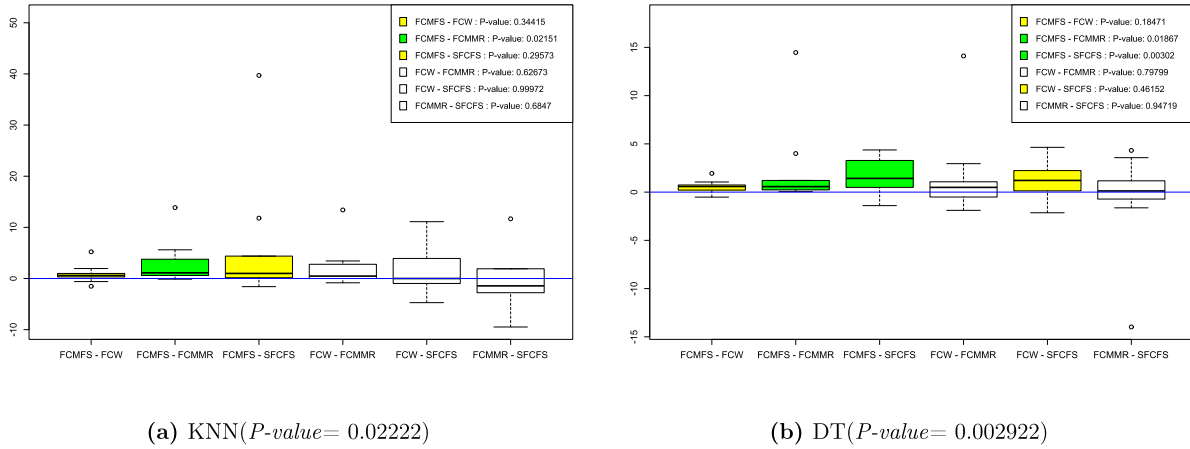


Fig. 2. The Boxplots of performance differences between FCMFS and three benchmarks marked by Post-hoc analysis for Friedman test.

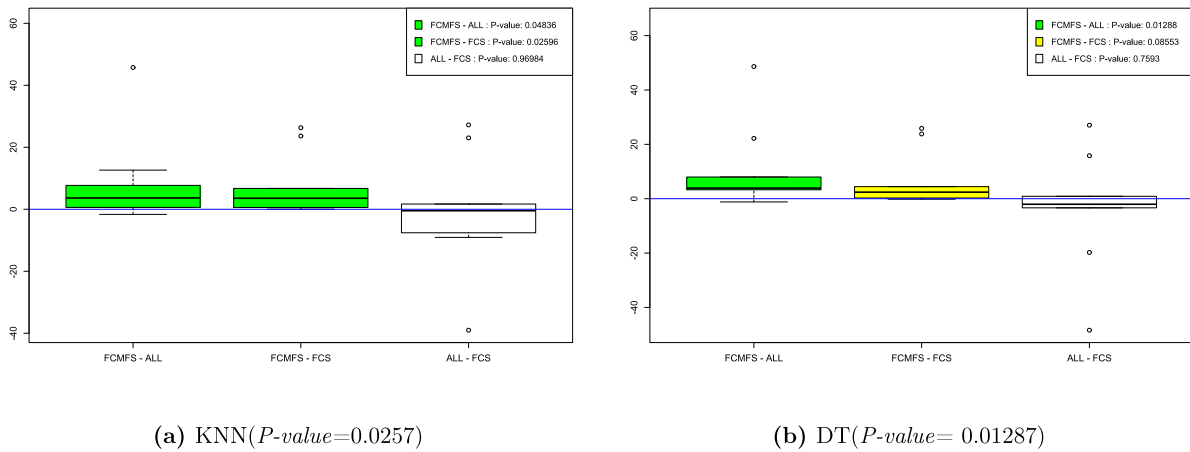


Fig. 3. The Boxplots of performance differences between FCMFS and two baselines marked by Post-hoc analysis for Friedman test.

algorithms on HillValley dataset achieves a little more than 50% accuracy, however, FCMFS can help KNN and DT obtain more than 98% accuracy. Moreover, FCMFS on Balance-scale dataset can increase A-KNN and A-DT by more than 10% and 20% respectively comparing with ALL. The A-KNN and A-DT are increased by 3% to 8% on Sonar, ionosphere and Thyroid datasets. Moreover, compared with original features, features constructed and selected by FCMFS have a much smaller number.

FCMFS is likewise much better than FCS on almost all the datasets in both KNN and DT learning algorithms, and FCMFS achieves much better classification performance than FCS on Wine and iris dataset. A-KNN and A-DT using FCMFS are increased by more than 23% on Wine and iris datasets comparing with using FCS. In addition, A-KNN and A-DT are significantly improved on Balance-scale, Thyroid, Sonar, HillValley and ionosphere datasets.

From the significance test shown in Fig. 3, we can see FCS cannot achieve significantly better classification performance than ALL. FCS can improve the classification performance of Balance-scale and HillValley datasets significantly, however FCS decreases the classification performance of iris and Wine datasets significantly. The experimental results show that single feature constructed by FCS does not have adequate discrimination ability. More constructed features need to be added to obtain better classification performance.

5.2.2. Comparisons between FCM, FS and ALL

The boxplots of performance differences between FCM, FS and ALL marked by Post-hoc analysis for Friedman test are shown

in Fig. 4. The Friedman test shows that there are significantly different between FCM, FS and ALL with $P\text{-value} = 0.03223$ in KNN learning algorithm and $P\text{-value} = 0.005903$ in DT learning algorithm. The Post-hoc analysis for Friedman test shows that FCM is significantly better than ALL in both KNN and DT learning algorithms, FS is better than ALL on most cases in KNN learning algorithm, and FS cannot improve the classification performance comparing with ALL in DT learning algorithm.

As shown in Table 4, FCM achieves significantly better classification performance than ALL in both KNN and DT learning algorithms on almost all the cases. FCM is much better than ALL on HillValley, Balance-scale, Sonar, ionosphere and Thyroid datasets. Compared with FCS, FCM can still improve the classification performance. The most obvious improvement can be seen on Wine and iris datasets. A-KNN and A-DT using FCS achieve a little more than 65% accuracy on Wine dataset, however, A-KNN and A-DT using FCM exceed 90%. The similar experimental results can be obtained on iris dataset. In addition, FCM can maintain or slightly increase classification performance comparing with FCS on other datasets.

According to the experimental results of Table 4, A-KNN using FS achieves 4.73% increase on sonar dataset, 1.96% increase on ionosphere dataset, 1.34% increase on iris dataset and 0.69% increase on Thyroid dataset comparing with using ALL. In other cases, A-KNN using FS is a little better or equivalent comparing with using ALL. A-DT using FS is equivalent comparing with using original features on most cases except for a slightly reduction on sonar dataset.

Table 4

The experimental results of different feature processing methods.

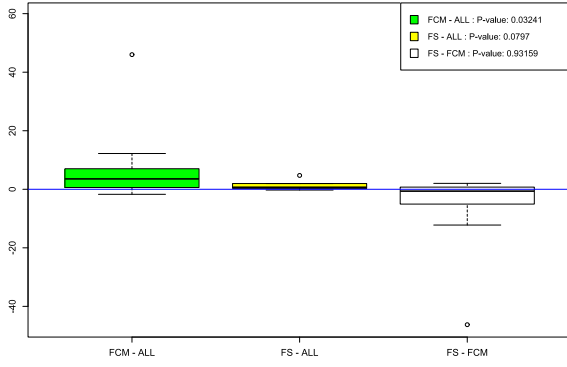
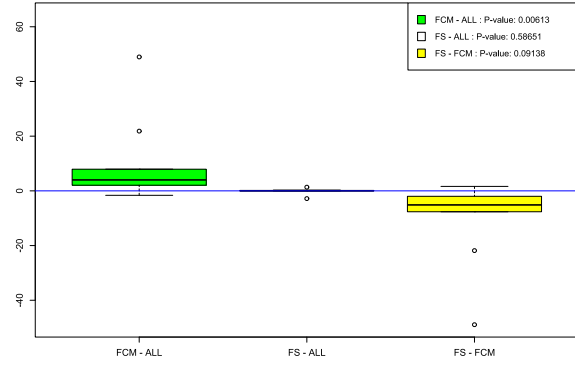
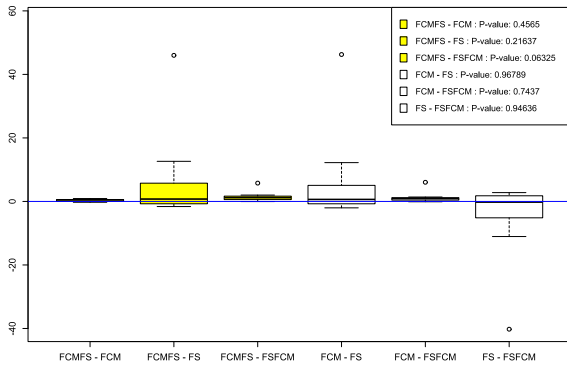
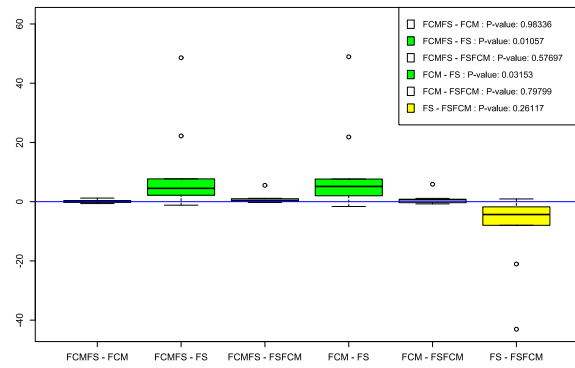
Dataset	Method	Subset	#F	B-KNN	A $\pm$ Std-KNN	B-DT	A $\pm$ Std-DT
Wine	ALL	F_o	13	100	95.07 $\pm$ 3.11	95.83	83.96 $\pm$ 5.62
	FCS	F_{cs}	1	83.33	67.85 $\pm$ 6.62	75.00	68.13 $\pm$ 4.57
	FCMFS	$(F_{cm})_s$	4.30	100	94.17 $\pm$ 3.62	97.92	91.94 $\pm$ 2.98
	FCM	F_{cm}	20	100	93.26 $\pm$ 6.62	97.92	91.88 $\pm$ 3.33
	FS	$(F_o)_s$	8.07	100	95.28 $\pm$ 3.40	95.83	84.24 $\pm$ 5.45
	FSFCM	$(F_o)_{scm}$	20	97.92	92.50 $\pm$ 3.86	97.92	92.22 $\pm$ 3.48
Sonar	ALL	F_o	60	78.33	68.83 $\pm$ 6.21	85	68.72 $\pm$ 10.01
	FCS	F_{cs}	1	88.33	71.72 $\pm$ 7.99	86.67	71.61 $\pm$ 8.51
	FCMFS	$(F_{cm})_s$	5.50	85	73.78 $\pm$ 6.84	86.67	72.61 $\pm$ 7.21
	FCM	F_{cm}	20	86.67	73.17 $\pm$ 8.92	86.67	72.72 $\pm$ 8.24
	FS	$(F_o)_s$	14.33	85	73.56 $\pm$ 6.04	86.67	65.89 $\pm$ 9.48
	FSFCM	$(F_o)_{scm}$	20	80	71.78 $\pm$ 5.04	80	71.61 $\pm$ 5.08
Wdbc	ALL	F_o	30	97.62	95.26 $\pm$ 1.49	96.43	92.22 $\pm$ 2.05
	FCS	F_{cs}	1	98.21	95.73 $\pm$ 1.61	98.21	95.58 $\pm$ 1.75
	FCMFS	$(F_{cm})_s$	5.03	98.21	96.31 $\pm$ 1.13	98.21	95.75 $\pm$ 1.50
	FCM	F_{cm}	20	98.21	96.19 $\pm$ 1.23	98.81	95.99 $\pm$ 1.83
	FS	$(F_o)_s$	8.37	97.62	95.56 $\pm$ 1.09	97.62	93.57 $\pm$ 2.03
	FSFCM	$(F_o)_{scm}$	20	98.21	95.77 $\pm$ 1.49	98.21	95.32 $\pm$ 1.60
HillValley	ALL	F_o	100	61.43	52.30 $\pm$ 3.28	50.41	50.38 $\pm$ 0.15
	FCS	F_{cs}	1	100	91.31 $\pm$ 11.71	99.72	98.75 $\pm$ 1.68
	FCMFS	$(F_{cm})_s$	3.28	99.72	98.04 $\pm$ 2.62	100	99.00 $\pm$ 0.80
	FCM	F_{cm}	20	100	98.30 $\pm$ 2.78	100	99.34 $\pm$ 0.54
	FS	$(F_o)_s$	7.87	62.26	52.04 $\pm$ 3.06	50.41	50.41 $\pm$ 0.00
	FSFCM	$(F_o)_{scm}$	20	99.72	92.27 $\pm$ 12.84	99.72	93.47 $\pm$ 13.19
ionosphere	ALL	F_o	34	88.24	81.18 $\pm$ 4.59	92.16	86.27 $\pm$ 3.86
	FCS	F_{cs}	1	93.14	88.76 $\pm$ 2.53	97.06	85.39 $\pm$ 10.04
	FCMFS	$(F_{cm})_s$	4.60	94.12	88.89 $\pm$ 2.72	97.06	89.54 $\pm$ 3.27
	FCM	F_{cm}	20	93.14	88.20 $\pm$ 3.04	96.08	88.33 $\pm$ 3.53
	FS	$(F_o)_s$	11.30	92.16	83.14 $\pm$ 3.95	93.14	86.34 $\pm$ 4.05
	FSFCM	$(F_o)_{scm}$	20	95.10	88.30 $\pm$ 3.05	95.10	89.08 $\pm$ 3.67
Balance-scale	ALL	F_o	4	91.67	86.67 $\pm$ 2.13	82.78	77.07 $\pm$ 2.6
	FCS	F_{cs}	1	100	95.74 $\pm$ 9.03	100	96.85 $\pm$ 8.99
	FCMFS	$(F_{cm})_s$	2.10	100	99.30 $\pm$ 3.21	100	99.26 $\pm$ 3.08
	FCM	F_{cm}	20	100	98.89 $\pm$ 3.01	100	98.91 $\pm$ 2.99
	FS	$(F_o)_s$	4	91.67	86.67 $\pm$ 2.13	82.78	77.07 $\pm$ 2.60
	FSFCM	$(F_o)_{scm}$	20	100	97.72 $\pm$ 2.76	100	98.12 $\pm$ 2.07
iris	ALL	F_o	4	100	94.81 $\pm$ 3.31	100	93.48 $\pm$ 3.80
	FCS	F_{cs}	1	86.67	71.78 $\pm$ 9.29	88.89	66.44 $\pm$ 11.45
	FCMFS	$(F_{cm})_s$	4.63	100	95.41 $\pm$ 2.92	97.78	92.30 $\pm$ 4.28
	FCM	F_{cm}	20	100	95.41 $\pm$ 2.92	100	91.85 $\pm$ 4.07
	FS	$(F_o)_s$	2	100	96.15 $\pm$ 2.06	100	93.48 $\pm$ 3.80
	FSFCM	$(F_o)_{scm}$	20	97.78	94.22 $\pm$ 3.12	100	92.56 $\pm$ 3.78
Thyroid	ALL	F_o	5	96.83	90.95 $\pm$ 2.90	96.83	89.37 $\pm$ 5.02
	FCS	F_{cs}	1	100	90.26 $\pm$ 7.04	96.83	89.58 $\pm$ 6.41
	FCMFS	$(F_{cm})_s$	4.47	98.41	94.60 $\pm$ 2.34	96.83	94.02 $\pm$ 2.95
	FCM	F_{cm}	20	100	94.50 $\pm$ 2.83	96.83	94.66 $\pm$ 1.99
	FS	$(F_o)_s$	3.63	96.83	91.64 $\pm$ 3.20	96.83	89.52 $\pm$ 4.43
	FSFCM	$(F_o)_{scm}$	20	98.41	93.60 $\pm$ 2.58	96.83	93.86 $\pm$ 2.45
WBC	ALL	F_o	9	98.07	96.38 $\pm$ 1.25	97.58	94.19 $\pm$ 1.82
	FCS	F_{cs}	1	98.07	94.69 $\pm$ 1.48	99.03	96.23 $\pm$ 1.69
	FCMFS	$(F_{cm})_s$	4.87	97.58	94.73 $\pm$ 1.26	98.55	96.05 $\pm$ 1.45
	FCM	F_{cm}	20	97.10	94.69 $\pm$ 1.39	98.55	95.67 $\pm$ 1.87
	FS	$(F_o)_s$	8.80	98.07	96.34 $\pm$ 1.22	97.58	94.19 $\pm$ 1.82
	FSFCM	$(F_o)_{scm}$	20	97.10	94.77 $\pm$ 1.14	98.07	95.52 $\pm$ 1.54

As shown in Table 4, through feature selection stage, redundant and irrelevant features are removed. $(F_o)_s$ selected by FS has a smaller number of features than original features. For example, on ionosphere dataset, the number of original features is 34, while the average number of $(F_o)_s$ in 30 independent runs is only 4.60 which is reduced to 11.5% of the original features. Moreover A-KNN of ionosphere dataset using FS is significantly better than that using ALL. The experimental results show that FS can select a smaller number of features and achieve better classification performance than using all features on some cases in KNN learning algorithms, and FS cannot achieve better classification performance but can remove redundant features in DT learning algorithm.

5.2.3. Comparisons between FCMFS and FCM, FS, FSFCM

This section compares the performances between FCMFS and FCM, FS, FSFCM respectively, and further verifies the effectiveness of FCMFS. The boxplots of performance differences between FCMFS, FCM, FS and FSFCM marked by Post-hoc analysis for Friedman test are shown in Fig. 5. Friedman test shows that the performance differences between FCMFS, FCM, FS and FSFCM are borderline significant in KNN learning algorithm with P -value=0.06296, and are significant in DT learning algorithm with P -value=0.01019.

5.2.3.1. Comparisons between FCMFS and FCM. As shown in Post-hoc analysis for Friedman test marked in Fig. 5, FCMFS is similar to FCM in both KNN and DT learning algorithms. According to Table 4, FCMFS on seven out of nine datasets achieves slightly

(a) KNN(P -value=0.03223)(b) DT(P -value= 0.005903)**Fig. 4.** The Boxplots of performance differences between FCM, FS and ALL marked by Post-hoc analysis for Friedman test.(a) KNN(P -value=0.06296)(b) DT(P -value=0.01019)**Fig. 5.** The Boxplots of performance differences between FCMFS, FCM, FS and FSFCM marked by Post-hoc analysis for Friedman test.

better classification performance than FCM in KNN learning algorithm. FCMFS on five out of nine datasets achieves better classification performance than FCM in DT learning algorithm and the differences between FCMFS and FCM are very small. However, the number of features produced by FCMFS is reduced to around 5 on all the datasets. So, our proposed FCMFS can achieve the goal of maintaining classification performance as well as reducing redundancy.

To further verify the effectiveness of feature selection method using GP, we compared with another variance where PSO is used instead of GP as the search algorithm. This feature selection method is named as FCMPSOFS. The same correlation-based method shown in Formula 3 is chosen as the fitness function. The standard continuous PSO is used. Parameters are set according to common setting proposed by [47]. The population size is 30. The maximum iteration is 100. The inertia weight $w = 0.7298$. The acceleration constants $c_1 = c_2 = 1.49618$. The threshold θ was set as 0.6 to determine whether a feature is selected or not. The experimental results are shown in Table 5.

As shown in Table 5, the classification performances of FCMFS and FCMPSOFS are very similar, which shows that PSO and GP, as two effective global search algorithms, can search for the optimal solutions. However, FCMFS can search for the feature subset with a smaller number of features, which is half or less of the number of features of FCMPSOFS. The threshold θ is an important parameter to impact the number of selected features in feature selection methods using PSO, and this parameter depends on specific datasets. The comparisons between FCMFS and FCMPSOFS

verify that FCMFS is effective to obtain equivalent classification performance with a smaller number of features.

5.2.3.2. Comparisons between FCMFS and FS. According to Post-hoc analysis for Friedman test in Fig. 5, FCMFS achieves better classification performance than FS on most cases in KNN learning algorithm, and achieves significantly better classification performance than FS in DT learning algorithm. The obvious example is HillValley dataset. The original features on HillValley dataset have weak discrimination ability. Feature selection methods only selects effective features, and does not create new features. So feature selection methods cannot achieve significantly higher classification performance than using original features on datasets like HillValley. Xue (2013) [1] used PSO-based feature selection method on HillValley dataset and also achieved more than 50% accuracy on KNN learning algorithm. However, feature construction method can construct new higher-level features, and sometimes can increase the classification performance significantly. FCMFS achieves more than 98% accuracy in both KNN and DT learning algorithms on the HillValley dataset. FCMFS can also achieve significantly better classification performance than FS on ionosphere, Balance-scale and Thyroid datasets. Moreover, as shown in Table 4, in most cases, the number of features produced by FCMFS is much smaller than that selected by FS, which shows that FCMFS can obtain better classification performance with smaller number of features than FS on the collected nine datasets.

The experimental results confirm that the discrimination ability of original features itself is not good enough in some cases.

Table 5

The experimental results of FCMFS and FCMPFS.

Dataset	Algorithm	#F	B-KNN	A $\pm$ Std-KNN	B-DT	A $\pm$ Std-DT
Wine	FCMFS	4.30	100	94.17 $\pm$ 3.62	97.92	91.94 $\pm$ 2.98
	FCMPFS	8.50	100	93.89 $\pm$ 5.04	97.92	91.81 $\pm$ 4.13
Sonar	FCMFS	5.50	85	73.78 $\pm$ 6.84	86.67	72.61 $\pm$ 7.21
	FCMPFS	10.13	86.67	74.50 $\pm$ 6.87	86.67	72.67 $\pm$ 8.18
Wdbc	FCMFS	5.03	98.21	96.31 $\pm$ 1.13	98.21	95.75 $\pm$ 1.50
	FCMPFS	9.83	98.21	96.11 $\pm$ 1.36	98.21	95.89 $\pm$ 1.78
HillValley	FCMFS	3.28	99.72	98.04 $\pm$ 2.62	100	99.00 $\pm$ 0.80
	FCMPFS	10.63	100	98.33 $\pm$ 2.79	100	98.96 $\pm$ 0.82
ionosphere	FCMFS	4.60	94.12	88.89 $\pm$ 2.72	97.06	89.54 $\pm$ 3.27
	FCMPFS	9.67	93.14	88.30 $\pm$ 2.76	97.06	89.67 $\pm$ 3.27
Balance-scale	FCMFS	2.10	100	99.30 $\pm$ 3.21	100	99.26 $\pm$ 3.08
	FCMPFS	12.63	100	99.43 $\pm$ 3.15	100	98.91 $\pm$ 2.99
iris	FCMFS	4.63	100	95.41 $\pm$ 2.92	97.78	92.30 $\pm$ 4.28
	FCMPFS	9.83	100	96.22 $\pm$ 2.64	100	91.85 $\pm$ 4.27
Thyroid	FCMFS	4.47	98.41	94.60 $\pm$ 2.34	96.83	94.02 $\pm$ 2.95
	FCMPFS	11.07	98.41	94.60 $\pm$ 2.64	96.83	94.34 $\pm$ 2.61
WBC	FCMFS	4.87	97.58	94.73 $\pm$ 1.26	98.55	96.05 $\pm$ 1.45
	FCMPFS	9.27	97.58	94.73 $\pm$ 1.54	98.55	95.68 $\pm$ 1.81

Feature selection method sometimes cannot obtain significantly promising classification performance. Sometimes GP can construct more informative features on the basis of original features and can achieve particular better classification performance than original features. We declare that the classification performance of FCMFS are not consistently better than that of FS, and which method is more effective depends on the properties of datasets. In practical applications, it is better to use some methods to evaluate the properties of the dataset in order to decide how to process original features and obtain better feature subsets.

5.2.3.3. Comparisons between FCMFS and FSFCM. The Post-hoc analysis for Friedman test in Fig. 5 shows that FCMFS is a little better than FSFCM on most cases in KNN learning algorithm, and the experimental results in Table 4 show that FCMFS achieves better classification performance than FSFCM on most cases in DT learning algorithm. A-KNN using FCMFS is 5.77% increase on HillValley dataset, 2% increase on Sonar dataset and 1.67% increase on Wine dataset, and in other cases, FCMFS slightly improve the classification performance comparing with FSFCM except for a slightly reduction on WBC dataset. According to Table 4, A-DT using FCMFS achieves 5.53% increase on HillValley dataset, 1.14% increase on Balance-scale dataset and 1% increase on Sonar dataset, and in other cases, FCMFS is a little better than FSFCM except for a little reduction on Wine and iris datasets. Moreover, FCMFS produces a smaller number of features comparing with FSFCM. The experimental results show that the order of FS and FCM has a great impact on the experimental results. FCMFS achieves better classification performance in most cases and produces smaller number of features than FSFCM.

The purpose of feature selection is to remove redundant and irrelevant features. These features are not very useful for classification. However, maybe these features help feature construction. For example, there are totally 60 features in Sonar dataset. Through feature selection stage, only 13 features are retained, that is, $(F_0)_s = (F_8, F_9, F_{10}, F_{11}, F_{12}, F_{20}, F_{35}, F_{36}, F_{43}, F_{46}, F_{48}, F_{50}, F_{51})$. The optimal solution produced by FSFCM generated from $(F_0)_s$ is as follows.

$$F_{c1} = (F_{51} * F_{12}) * (((F_{10} * F_9) * (F_{46} * F_{48})) * (((F_{12} * F_{35}) + (F_{10} + (F_{50} * F_{51}))) - ((F_{35} - F_9) - (F_8 * F_{10}))))).$$

The optimal solution produced by FCMFS is as follows.

$$F_{c2} = F_9 + ((F_{16} * F_{43}) + (((((F_2 + F_{10}) + (F_3 * F_{39})) + (((F_{33} - F_{25}) * F_{43}) * (F_{48} * (((F_{33} + (F_3 * F_{39})) * ((F_2 + F_{10}) + (F_2 + F_{10})))))) + (((((F_2 + F_{10}) + (F_3 * F_{39})) * (F_{39} + F_7)) + (F_3 * F_{39})) + (F_3 * F_{39})) + (((((F_2 + F_{10}) + (F_3 * F_{39})) * ((F_2 + F_{10}) +$$

$$F_7))) + (F_3 * F_{39})) * (F_{11} + F_{52})) * (F_7 * (((F_{33} - F_{25}) * F_{43}) + (F_2 + F_{10})))))) + ((F_3 * F_{39}) * ((F_{39} + (F_3 * F_{39})) * F_{45}))) - F_0)).$$

The above expression F_{c2} contains F_0, F_2, F_3, F_7 , etc features, which do not appear in F_{c1} . Experimental results show that F_{c2} can obtain better classification performance than F_{c1} . This example confirms that GP can easily construct effective features from original features, though the dimensionality of original features in some datasets is high and some original features have bad discrimination ability. Combinations of weak features with other features may construct more effective features. FSFCM algorithm removes some seemingly weak features at the first feature selection stage, and this may be the reason for causing worse experimental results than FCMFS.

5.3. The parameter setting of β in FCM

The FCM algorithm is to construct top β features according to the ranking of individuals during a GP run. How to set the value of the parameter β is a problem. If the β value is too large, a large number of redundant features will be constructed; if the β value is too small, on the special case of $\beta = 1$, the effect of the FCM algorithm is same as the FCS algorithm. To verify appropriate β value, first the parameter β was set to 1, i.e., single constructed feature was constructed and that was the experimental result of FCS, then the value of parameter β was increased by 1 in sequence, until the maximum value was 50. The experimental results on typical four datasets are shown in Fig. 6.

As shown in Fig. 6, for Wine, iris and Thyroid datasets, the general trend is that the accuracy increases with the increase of the parameter value β . When the parameter value β exceeds certain values, the accuracy will keep stable. For sonar dataset, the accuracy oscillatory rises with the number of constructed features increasing. The accuracy does not drastically change, when the parameter β exceeds certain values. From the experimental results in Fig. 6, we can roughly see that several constructed features are enough to obtain satisfactory classification performance, which are consistent to the experimental results of FCMFS shown in Table 4. The experimental results on other datasets follow the above rules.

Overall considering the experimental results of all the nine datasets, according to the principles of removing redundant features as far as possible without accuracy loss, we set the parameter value $\beta = 20$. Actually, for most datasets, there are some redundant features within the 20 constructed features. So,

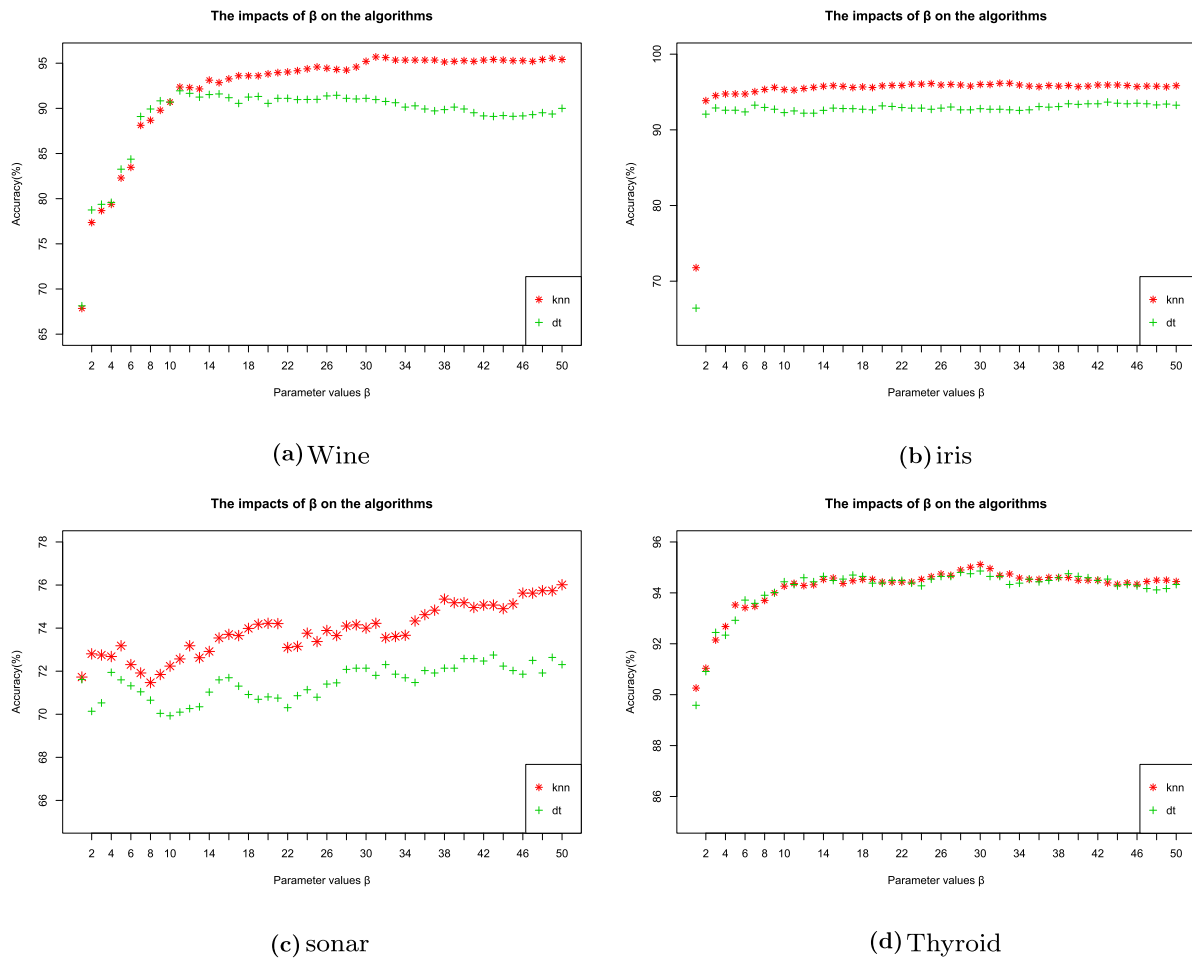


Fig. 6. The impact of parameter β on experimental results.

a feature selection stage is needed to remove these redundant features, and to obtain effective feature subsets. The above experiments show that our proposed FCMFS is effective to not only achieve satisfactory classification performance but also obtain as few features as possible.

5.4. Further discussions

In this paper, we investigated five filter-based feature processing methods including FS, FCS, FCM, FSFCM and FCMFS. Experiments on nine datasets show that the five feature processing methods are all effective. For filter-based feature construction methods that use IGR as evaluation function, the multiple feature construction method (FCM) can achieve significantly better classification performance than the single feature construction method (FCS). Experimental results show that single constructed feature does not have enough discrimination ability for classification, and multiple feature construction methods can achieve significantly better classification performance than single feature construction method.

The disadvantage of our multiple feature construction method is that there are a lot of redundant features. So then we used a GP-based feature selection method (FS) to select effective features and remove redundant features. The feature selection method uses correlation as an evaluation function and employs GP as the search algorithm. The experimental results show that FS can obtain comparable or better classification performance and only select a smaller number of features than using original features. GP is a global search algorithm. We made a preliminary

comparison between feature selection using PSO and GP search algorithm, and found that GP could obtain a smaller number of features than PSO. Due to GP's flexible representation, GP can be used for feature construction, feature selection and classifier. This is the advantage of GP over other EC algorithms that only used for searching. It is an interesting work to use GP to evolve a simpler solution with feature construction and feature selection, which has good discrimination ability and interpretation for classification.

The proposed FCMFS uses two separate GP runs. The first is feature construction stage that constructs multiple features, and the second is feature selection stage that selects effective features from the constructed features. Experiments show that FCMFS can maintain the classification performance with smaller number of features comparing with FCM. Then we investigated FSFCM that swapped the order of the two stages of FCMFS and found that FCMFS achieved better performance than FSFCM. For GP-based feature construction method, the original features are the inputs of GP tree, and the root of the GP tree is the output. For the nine datasets with at most tens to hundred of dimensions, GP can automatically select effective original features as inputs, and discard the irrelevant features. Moreover, GP can discover hidden relationships between features and construct more effective features. However, if feature selection is carried out first, potentially useful original features may be discarded, and these features may be helpful for feature construction. For example, FCMFS is very effective on the HillValley dataset, while FSFCM achieves much worse performance than FCMFS. For high-dimensional applications with thousands of dimensions, it could

be difficult for GP to search for effective original features for inputs, which may affect the effect of feature construction. In high-dimensional applications, how to combine feature selection methods to construct more effective features is one of our future research directions.

Experiments on nine datasets show that FCM and FCMFS are significantly more effective than FS. In some classification applications, available original features do not have adequate discrimination ability. In this case, feature selection methods cannot achieve ideal classification results. However, feature construction methods can create new higher-level features from original ones. Sometimes, feature construction methods can improve the classification performance significantly. For example, our feature construction methods can achieve significantly better classification performance on HillValley and Balance-scale datasets than using original features. However, feature selection methods are far less effective than feature construction methods on Balance-scale and HillValley datasets. Feature construction methods are not always more effective than feature selection methods, and which methods are more effective depends on the properties of datasets. Therefore, it may be interesting to estimate the characteristics of datasets to decide whether to use feature selection or feature construction.

6. Conclusions and future work

This paper proposes a filter-based feature construction and feature selection approach using GP(FCMFS) that is divided into two stages, i.e., first using multiple feature construction approach to store top individuals(FCM) and then using feature selection approach to select effective feature subset(FS). The experiments on nine datasets show that FCM and FS can obtain better performance than original features. FCMFS can maintain the classification performance with a smaller number of features comparing with FCM and FCMFS can achieve better performance than FS on the nine datasets. Further investigations on changing the order of FCM and FS show that FCMFS can retain effective features with discrimination ability and can achieve better classification performance than FSFCM. Compared with three state-of-art techniques, our proposed FCMFS can obtain better classification performance in most cases.

Feature construction and feature selection are two effective feature processing approaches. FCMFS proposed in this paper combines feature construction and feature selection approaches and can obtain better performance than feature construction and feature selection only. Different datasets are suitable for different feature processing approaches. So, some work needs to be done to estimate the properties of datasets in order to decide how to preprocess the features of original datasets before classification. In this work, both FCM and FS use standard GP. Our future work will focus on improvement of standard GP to further improve the classification performance. Our work focuses on datasets at most tens to hundred of dimensions. How to combine feature selection methods to construct more effective features in high-dimensional applications with thousands of dimensions is also our future research point.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit authorship contribution statement

Jianbin Ma: Conceptualization, Methodology, Software, Validation, Supervision, Investigation, Writing - original draft. **Xiaoying Gao:** Writing - review & editing, Formal analysis, Visualization, Validation.

Acknowledgments

This work is supported by Hebei Agricultural University (No. ZD201702, No. LG201707) and Hebei Provincial Department of Human Resources and Social Security, China (No. CN201709).

References

- [1] B. Xue, M. Zhang, W. Browne, Particle swarm optimization for feature selection in classification: A multi-objective approach, *IEEE Trans. Cybern.* 43 (6) (2013) 1656–1671.
- [2] B. Tran, B. Xue, M. Zhang, Genetic programming for feature construction and selection in classification on high-dimensional data, *Memet. Comput.* 8 (1) (2016) 3–15.
- [3] F.E.B. Otero, M.M.S. Silva, A.A. Freitas, J.C. Nievola, Genetic programming for attribute construction in data mining, in: *Proceedings of 2003 European Conference on Genetic Programming*, Springer-Verlag, 2003, pp. 384–393.
- [4] I.A. Gheyas, L.S. Smith, Feature subset selection in large dimensionality domains, *Pattern Recognit.* 43 (1) (2010) 5–13.
- [5] P. Bugata, P. Drotlár, Weighted nearest neighbors feature selection, *Knowl.-Based Syst.* 163 (2019) 749–761.
- [6] B. Tran, *Evolutionary Computation for Feature Manipulation in Classification on High-Dimensional Data* (Ph.D. thesis), Victoria University of Wellington, 2018.
- [7] M. Muharram, G.D. Smith, Evolutionary constructive induction, *IEEE Trans. Knowl. Data Eng.* 17 (11) (2005) 1518–1528.
- [8] E. Hart, K. Sim, B. Gardiner, K. Kamimura, A hybrid method for feature construction and selection to improve wind-damage prediction in the forestry sector, in: *Genetic and Evolutionary Computation Conference*, 2017, pp. 1121–1128.
- [9] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, I.H. Witten, The WEKA data mining software: an update, *Acm Sigkdd Explor. Newslett.* 11 (1) (2009) 10–18.
- [10] K. Neshatian, M. Zhang, P. Andreae, A filter approach to multiple feature construction for symbolic learning classifiers using genetic programming, *IEEE Trans. Evol. Comput.* 16 (5) (2012) 645–661.
- [11] M.G. Smith, L. Bull, Feature construction and selection using genetic programming and a genetic algorithm, in: *Proceedings of 2003 European Conference on Genetic Programming*, 2003, pp. 229–237.
- [12] J.R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*, MIT Press, 1992.
- [13] W. Banzhaf, P. Nordin, R.E. Keller, F.D. Francone, *Genetic programming: An introduction to the automatic evolution of computer programs and its applications*, *J. Combin. Theory* 71 (2) (1998) 130–143.
- [14] K. Neshatian, *Feature Manipulation with Genetic Programming* (Ph.D. thesis), Victoria University of Wellington, 2010.
- [15] S. Ahmed, M. Zhang, L. Peng, Feature selection and classification of high dimensional mass spectrometry data: A genetic programming approach, in: *Proceedings of 2013 European Conference on Evolutionary Computation, Machine Learning and Data Mining in Bioinformatics*, 2013, pp. 43–55.
- [16] D.Y. Harvey, M.D. Todd, Automated feature design for numeric sequence classification by genetic programming, *IEEE Trans. Evol. Comput.* 19 (4) (2014) 474–489.
- [17] R.A. Davis, A.J. Charlton, S. Oehlschlager, J.C. Wilson, Novel feature selection method for genetic programming using metabolomic ¹H NMR data, *Chemometr. Intell. Lab. Syst.* 81 (1) (2006) 50–59.
- [18] M.A. Muharram, G.D. Smith, Evolutionary feature construction using information gain and gini index, in: *Proceedings of 2004 European Conference on Genetic Programming*, 2004, 379–388.
- [19] H. Guo, A.K. Nandi, Breast cancer diagnosis using genetic programming generated feature, *Pattern Recognit.* 39 (5) (2006) 980–987.
- [20] H. Guo, Q. Zhang, A.K. Nandi, Feature extraction and dimensionality reduction by genetic programming based on the Fisher criterion, *Expert Syst.* 25 (5) (2008) 444–459.
- [21] K. Krawiec, Genetic programming-based construction of features for machine learning and knowledge discovery tasks, *Genet. Progr. Evol. Mach.* 3 (4) (2002) 329–343.
- [22] S. Ahmed, M. Zhang, L. Peng, B. Xue, Multiple feature construction for effective biomarker identification and classification using genetic programming, in: *Proceedings of the 2014 Conference on Genetic and Evolutionary Computation*, pp. 249–256.
- [23] Y. Lin, B. Bhanu, Evolutionary feature synthesis for object recognition, *IEEE Trans. Syst. Man Cybern. C* 35 (2) (2005) 156–171.
- [24] B. Bhanu, K. Krawiec, Coevolutionary construction of features for transformation of representation in machine learning, in: *Proceedings of 2002 Genetic and Evolutionary Computation Conference*, 2002, pp. 249–254.
- [25] K. Krawiec, Generative learning of visual concepts using multiobjective genetic programming, *Pattern Recognit. Lett.* 28 (16) (2007) 2385–2400.

- [26] K. Krawiec, B. Bhanu, Visual learning by evolutionary and coevolutionary feature synthesis, *IEEE Trans. Evol. Comput.* 11 (5) (2007) 635–650.
- [27] J. Ma, G. Teng, A hybrid multiple feature construction approach using genetic programming, *Appl. Soft Comput.* 80 (2019) 687–699.
- [28] H. Firpi, E. Goodman, J. Echaz, On prediction of epileptic seizures by means of genetic programming artificial features, *Ann. Biomed. Eng.* 34 (3) (2006) 515–529.
- [29] B. Tran, M. Zhang, B. Xue, Multiple feature construction in classification on high-dimensional data using GP, in: *Proceedings of 2016 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2016, pp. 1–8.
- [30] W.M. Spears, K.A.D. Jong, D.B. Fogel, H.D. Garis, An overview of evolutionary computation, in: *Proceedings of 1993 European Conference on Machine Learning*, pp. 442–459.
- [31] J.R. Koza, D. Andre, F.H. Bennett, M.A. Keane, Genetic programming III - Darwinian invention and problem solving, *IEEE Trans. Evol. Comput.* 7 (4) (2002) 451–453.
- [32] K. Lu, W. Zhou, G. Zeng, Y. Zheng, Constrained population extremal optimization-based robust load frequency control of multi-area interconnected power system, *Int. J. Electr. Power Energy Syst.* 105 (2019) 249–271.
- [33] K. Lu, W. Zhou, G. Zeng, W. Du, Design of PID controller based on a self-adaptive state-space predictive functional control using extremal optimization method, *J. Franklin Inst. B* 355 (5) (2018) 2197–2220.
- [34] G.Q. Zeng, J. Chen, Y.-X. Dai, L.-M. Li, C.-W. Zheng, M.-R. Chen, Design of fractional order PID controller for automatic regulator voltage system based on multi-objective extremal optimization, *Neurocomputing* 160 (2015) 173–184.
- [35] S. Saremi, S. Mirjalili, A. Lewis, A.W.C. Liew, J.S. Dong, Enhanced multi-objective particle swarm optimisation for estimating hand postures, *Knowl.-Based Syst.* 158 (2018) 175–195.
- [36] N. Dong, C. Dai, An improvement decomposition-based multi-objective evolutionary algorithm using multi-search strategy, *Knowl.-Based Syst.* 163 (2019) 572–580.
- [37] P.G. Espejo, S. Ventura, F. Herrera, A survey on the application of genetic programming to classification, *IEEE Trans. Syst. Man Cybern. C* 40 (2) (2010) 121–144.
- [38] J.R. Quinlan, C4.5: programs for machine learning, 1993, Morgan Kaufmann series in machine learning.
- [39] C.J. Mantas, J. Abellán, Credal-C4.5: Decision tree based on imprecise probabilities to classify noisy data, *Expert Syst. Appl.* 41 (10) (2014) 4625–4637.
- [40] M.G. Smith, L. Bull, Genetic programming with a genetic algorithm for feature construction and selection, *Genet. Program. Evol. Mach.* 6 (3) (2005) 265–281.
- [41] M.A. Hall, Correlation-Based Feature Selection for Machine Learning (Ph.D. thesis), The University of Waikato, 1999.
- [42] M.A. Hall, Correlation-based feature selection for discrete and numeric class machine learning, in: *Proceedings of Seventeenth International Conference on Machine Learning*, ACM, 2000, pp. 359–366.
- [43] M.A. Hall, L.A. Smith, Feature selection for machine learning: Comparing a correlation-based filter approach to the wrapper, in: *Proceedings of the Twelfth International Florida Artificial Intelligence Research Society Conference*, 1999.
- [44] S.D. Asit K.D.as, A. Ghosh, Ensemble feature selection using bi-objective genetic algorithm, *Knowl.-Based Syst.* 123 (2017) 116–127.
- [45] W.E. Chih Fong Tsai, C.Y. Chu, Genetic algorithms in feature and instance selection, *Knowl.-Based Syst.* 39 (2013) 240–247.
- [46] M.S.H. Hojat Ghimatgar, A. Aarabi, An improved feature selection algorithm based on graph clustering and ant colony optimization, *Knowl.-Based Syst.* 159 (2018) 270–285.
- [47] B. Xue, M. Zhang, W.N. Browne, Particle swarm optimisation for feature selection in classification: novel initialisation and updating mechanisms, *Appl. Soft Comput.* 18 (2014) 261–276.
- [48] K. Neshatian, M. Zhang, Genetic programming for feature subset ranking in binary classification problems, in: *Proceedings of 2009 European Conference on Genetic Programming*, 2009, pp. 121–132.
- [49] K. Neshatian, M. Zhang, Pareto front feature selection: using genetic programming to explore feature space, in: *Proceedings of 2009 Genetic and Evolutionary Computation Conference*, 2009, pp. 1027–1034.
- [50] K. Neshatian, M. Zhang, Improving relevance measures using genetic programming, in: *Proceedings of 2012 European Conference on Genetic Programming*, 2012, pp. 97–108.
- [51] B. Xue, M. Zhang, W.N. Browne, X. Yao, A survey on evolutionary computation approaches to feature selection, *IEEE Trans. Evol. Comput.* 20 (4) (2016) 606–626.
- [52] D.P. Muni, N.R. Pal, J. Das, Genetic programming for simultaneous feature selection and classifier design, *IEEE Trans. Syst. Man Cybern. B* 36 (1) (2006) 106–117.
- [53] I. Sandin, G. Andrade, F. Viegas, D. Madeira, Aggressive and effective feature selection using genetic programming, in: *Proceedings of 2012 IEEE Congress on Evolutionary Computation*, 2012, pp. 1–8.
- [54] K. Neshatian, M. Zhang, Using genetic programming for context-sensitive feature scoring in classification problems, *Connect. Sci.* 23 (3) (2011) 183–207.
- [55] V. Venkatraman, A.R. Dalby, Z.R. Yang, Evaluation of mutual information and genetic programming for feature selection in qsar, *J. Chem. Inf. Comput. Sci.* 44 (5) (2004) 1686–1692.
- [56] H. Guo, L.B. Jack, A.K. Nandi, Feature generation using genetic programming with application to fault classification, *IEEE Trans. Syst. Man Cybern. B* 35 (1) (2005) 89–99.
- [57] U. Kamath, J.K. De, A. Shehu, Effective automated feature construction and selection for classification of biological sequences, *PLoS One* 9 (7) (2014) e99982.
- [58] C.D. Rosin, R.K. Belew, New methods for competitive coevolution, *Evol. Comput.* 5 (1) (2014) 1–29.
- [59] D. Dheeru, E. Karra Taniskidou, UCI Machine Learning Repository, University of California, Irvine, School of Information and Computer Sciences, 2017, URL <http://archive.ics.uci.edu/ml>.
- [60] B. Xue, M. Zhang, W.N. Browne, Multi-objective particle swarm optimisation(PSO) for feature selection, in: *Proceedings of 2012 Conference on Genetic and Evolutionary Computation*, 2012, pp. 81–88.
- [61] S. Luke, ECJ then and now, in: *Proceedings of the 2017 Genetic and Evolutionary Computation Conference Companion*, 2017, pp. 1223–1230.
- [62] M. Hollander, D.A. Wolfe, *Nonparametric Statistical Methods*, second ed., Wiley-Interscience, 1999.
- [63] R. Development Core Team, R: A Language and Environment for Statistical Computing, R Foundation for Statistical Computing, Vienna, Austria, ISBN: 3-900051-07-0, 2010, <http://www.R-project.org>.
- [64] W.A. Robert McGill, Variations of box plots, *Amer. Statist.* 32 (1) (1978) 12–16.